Pipeline Reality Checker: Product Brief

Abhijith K · Updated 15 October 2026

Pipeline Reality Checker tells a sales manager which open deals are likely to slip, why, and what to do next, using fixed rules for the score and AI only to read deal notes. It exists because pipelines report what reps entered, not what buyers are doing.

Problem Statement

Sales pipelines look healthier than they are because a CRM stage records what the rep last updated, not what the buyer is doing.

A deal can sit in “Negotiation” for six weeks with no reply from the customer and still count toward the forecast at full value. Sales managers review 40 to 60 open deals a week, mostly by asking reps how things are going. Reps are optimistic by nature, so the review confirms the pipeline instead of testing it.

The warning signs are usually already in the data: no activity for weeks, a deal stuck in one stage far longer than normal, a close date that has quietly passed, or a note saying the decision-maker is on leave. Nobody connects these signals in one place. By the time a deal visibly slips, it is too late in the quarter to recover it.

The cost shows up as missed forecasts, manager time spent on the wrong deals, and interventions that come too late.

User Persona

Priya Nair is a regional sales manager at a mid-sized B2B ingredients supplier, and she owns a forecast she can’t fully trust.

Attribute	Detail
Role	Regional Sales Manager, 9 years in B2B sales
Team	6 account managers
Pipeline	45 to 60 open deals, ₹5 lakh to ₹2 crore each
Sales cycle	60 to 120 days, 3 to 5 people involved on the buyer side
Tools	CRM (Zoho or Salesforce), weekly Excel export, WhatsApp and email with buyers
Weekly ritual	Friday pipeline review, then a forecast call with the sales head on Monday

What she wants: to know on Friday which 5 to 8 deals need her attention next week, and what to do about each.

What frustrates her:

- Reps report deals as “on track” until the week they’re lost
- The CRM shows stage and value, but not whether anything is actually moving
- She doesn’t have time to read every deal’s notes before the review
- Generic dashboards flag too many deals, so she ignores them

What success looks like: fewer surprises at quarter end, and a forecast she can defend to her boss.

JTBD Statement

> *When I review my pipeline at the end of the week, I want to know which deals are actually at risk and why, so I can act before they slip.*

Trigger moment: Friday afternoon. The pipeline total looks healthy, but last month’s revenue came in short and Priya can’t tell which of this month’s deals will do the same.

Current workaround: she exports the CRM to Excel, sorts by close date, and asks each rep about their biggest deals. This takes about two hours and depends entirely on what reps choose to tell her.

How we’ll know the job is done:

- She gets a short, ranked list of at-risk deals in under a minute
- Every flag shows the evidence behind it, so she can check it in seconds
- Each flag comes with one specific next action she can assign on Monday
- Over time, flagged deals slip more often than unflagged ones (tracked through stored outcomes)

Why AI?

Most of this problem is arithmetic, so most of the product is rules. AI handles only the part rules can't: reading free-text deal notes.

Task	Done by	Why
Days since last activity	Rules	A date subtraction. An LLM could get it wrong and gains nothing
Days in current stage vs. normal for that stage	Rules	Needs a fixed threshold so results are consistent week to week
Close date already passed	Rules	Simple yes/no check
Deal size unusually large for its stage	Rules	Comparison against the rest of the pipeline
Risk score (0 to 100)	Rules	Must be explainable and repeatable
Reading notes like "CFO on leave till Nov" or "also talking to a competitor"	AI	Notes are unstructured and written differently by every rep
Writing the reason in plain English	AI	Turns rule hits and note signals into one readable sentence
Suggesting one specific next action	AI	Depends on the deal's context, which rules can't capture

Why not rules only? The strongest risk signals often sit in the notes: no decision-maker involved, budget on hold, a competitor in the picture. Keyword matching misses most of these because reps phrase them in many ways.

Why not AI only? An LLM would be inconsistent on dates and thresholds, would cost more per run, and would be tempted to find risk everywhere. That is the false-positive problem this product must avoid.

The rule that ties them together: the rules decide whether a deal is flagged. The AI can explain a flag, add a note-based signal, or lower a flag when the notes explain the silence. Every AI signal must quote the notes, and AI evidence alone can raise a deal to Watch at most; reaching At Risk always takes a rule.

System Flow & Logic Layer

Every deal passes through the same 10 steps, and a deal's tier is fixed by rules before the AI ever sees it.

Input fields

Field	Type	Why it's needed
Deal ID	Text	Links each run and outcome to the same deal
Account name	Text	Shown to the manager; replaced with Deal ID before the AI call
Owner (rep)	Text	Lets the manager assign the next action
Stage	Fixed list: Discovery, Proposal, Negotiation, Contract Review	Thresholds depend on stage
Deal size (₹)	Number, above 0	Ranks risk by money at stake
Last activity date	Date, not in the future	Inactivity check
Stage entered date	Date, not after last activity	Stage stagnation check
Expected close date	Date	Passed-close-date check
Notes	Free text, optional	Read by the AI for signals rules can't see

The assignment requires stage, last activity, notes and deal size. I added stage entered date because stagnation can't be measured without it, plus close date and owner.

The pipeline

1. **Structured input form:** add deals one at a time or upload a CSV.
2. **Input validation:** reject rows with missing fields, future dates, unknown stages or a size of zero; tell the user which field to fix.
3. **Logic layer:** compute days inactive, days in stage and close-date status; apply the scoring rules below.
4. **Prompt construction:** send only deals with notes or rule hits, with names masked and rule results attached.
5. **LLM API call:** one batched call to Groq (openai/gpt-oss-120b) at temperature 0.2, asking for JSON per deal.
6. **Output parsing:** check each deal's JSON separately; drop any reason that doesn't quote a note or name a rule.
7. **Confidence and flag check:** apply note-based points, recompute the tier and assign confidence.
8. **Database storage:** save the run, every deal's score, reasons and confidence.
9. **Structured UI output:** at-risk deals first, each with tier, reason, evidence, next action and confidence.
10. **Admin dashboard:** runs over time, flags by reason, overrides and flag accuracy from recorded outcomes.

Default stage thresholds (set in one place in the code)

Stage	Max days without activity	Max days in stage
Discovery	21	30
Proposal	14	30
Negotiation	10	21
Contract Review	14	30

Scoring rules

Signal	Source	Points
Close date has passed, deal still open	Rule	+50 (strong)
Inactive more than twice the stage limit	Rule	+50 (strong)
Inactive beyond the stage limit	Rule	+25
In stage beyond the stage limit	Rule	+25
Deal in the top 20% by size and inactive more than 7 days	Rule	+10
Note signal: no decision-maker, budget on hold, competitor, buyer delay	AI, must quote the note	+15 each, maximum +30
Note explains the silence with a dated next step	AI, must quote the note	-25

Tiers: Healthy below 25, Watch 25 to 49, At Risk 50 and above (score capped at 100).

This gives the false-positive safeguards real numbers. One standard rule puts a deal on Watch. At Risk needs two rule signals or one strong one. AI evidence alone can reach Watch at most, never At Risk.

Confidence

- **High:** rule signals and note evidence point the same way, and all fields are filled.
- **Medium:** rule signals only, because notes are empty or neutral.
- **Low:** notes contradict the rules, data is incomplete, or the AI call failed and results are rules-only.

Measuring whether it works

The dashboard compares each flag with the outcome the manager records later. Targets for the pilot: at least 60% of At Risk deals end up lost or slipped, and managers override fewer than 20% of flags. A high override rate means the thresholds are too tight.

Feature List + Value vs Effort

The MVP covers the full pipeline the brief requires; CRM integrations and alerts wait until the core logic has proven it flags the right deals.

#	Feature	What it does	Value	Effort	Plan
1	Deal input form + CSV upload	Enter deals one by one or upload a CRM export	High	Low	MVP
2	Input validation	Rejects missing fields, future activity dates, negative deal sizes, unknown stages	High	Low	MVP
3	Rules engine	Checks inactivity, stage stagnation, passed close date, size outliers	High	Medium	MVP
4	Risk score and tier	Scores 0 to 100 and labels each deal Healthy, Watch or At Risk	High	Low	MVP
5	AI note analysis	Reads notes, writes the reason and one next action	High	Medium	MVP
6	Confidence signal	High, Medium or Low per flag, based on how much evidence supports it	High	Low	MVP
7	Reasoning panel	Shows exactly which rules fired and which note text was used	High	Low	MVP
8	Saved analyses + outcome tracking	Stores every run; manager marks each deal Won, Lost or Slipped later	Medium	Medium	MVP
9	Admin dashboard	Runs over time, flags by reason, how often flagged deals actually slipped	Medium	Medium	MVP
10	Adjustable thresholds	Stage limits live in one place in the code and show on the dashboard; editing them from the app comes next	Medium	Low	Partly built
11	“Not at risk” override	Manager dismisses a flag with a reason; stored as feedback	Medium	Low	MVP
12	CRM integration	Pull deals directly from Salesforce, HubSpot or Zoho	High	High	Later
13	Weekly email or Slack digest	Sends the at-risk list every Friday	Medium	Medium	Later
14	Per-rep patterns	Shows which reps’	Medium	Medium	Later

		deals slip most often			

Features 12 to 14 matter for a real product, but they add integration work without testing the core question: does the tool flag the right deals?

Failure Mode Analysis

The biggest risk is flagging healthy deals: if Priya sees three false alarms in her first week, she stops opening the tool.

Key failure mode: false positives

Some healthy deals look quiet. A contract sitting with the buyer's legal team, a deal waiting on a scheduled board meeting, or a large enterprise deal with a naturally long cycle can all go weeks without activity.

Safeguards:

1. **Stage-specific thresholds.** 14 days of silence means something different in Discovery than in Contract Review.
2. **Two signals to reach At Risk.** One weak signal puts a deal on Watch, not At Risk. At Risk needs two signals, or one strong one such as a passed close date.
3. **Notes can clear a flag.** If the notes explain the silence ("with their legal, response expected 20 Oct"), the AI lowers the tier and says why.
4. **Three tiers, not a yes/no.** Watch gives a soft signal without crying wolf.
5. **Manager override is stored.** Each dismissal is saved with its reason, so thresholds can be tuned with real data.

Other failure modes

Failure mode	Example	Safeguard
False negative	Rep logs a call every week, but the deal hasn't moved stage in 60 days	Stage age is checked separately from activity
AI invents a reason	"Customer seems unhappy" with nothing in the notes to support it	Every note signal must quote the notes, and the code checks the quote word for word against the original. Claims that fail are dropped and shown as "AI claim dropped"; a summary with no evidence behind it is never shown
Generic next action	"Follow up with the client"	Prompt requires a named person or specific step; a rule-based fallback action replaces vague output
Bad input data	Last activity date in the future, missing stage	Validation rejects the row and tells the user which field to fix
LLM call fails or returns broken output	API timeout, malformed JSON, or the model being unavailable (this happened in testing: the first Groq model returned a 404 for my account)	One retry, then show rules-only results with a clear "AI analysis unavailable" label. In the real case the app kept working on rules alone and the banner named the error; switching models took one setting
Manager trusts the score blindly	Acts on a Low-confidence flag without checking	Confidence shown on every flag; reasoning panel always visible

AI Trade-offs

I chose consistency and explainability over clever-sounding analysis, because a sales manager acts on this output with real money at stake.

Trade-off	Choice	What I give up
Consistency vs. creativity	Low temperature (0.2) and a fixed output format	Less varied wording in reasons
Rules vs. AI for the score	Rules set most of the score; AI can add up to 30 points or remove 25, always with a	The AI can't catch a risk that appears in neither the data nor the notes

	quote from the notes	
Model size vs. cost	Groq's free tier running openai/gpt-oss-120b, an open-weight model; move to a paid tier or another provider only if testing shows poor note reading	Some nuance in messy or mixed-language notes, plus the free tier's rate limits
One call per deal vs. one batch call	Batch all deals with notes into one call	One bad response can affect several deals, so parsing checks each deal separately
Detail vs. speed	One reason and one action per deal, not a full write-up	Less context, but readable in a 10-minute review
Privacy vs. context	Send notes and stage data only; buyer company names replaced with Deal IDs	The AI can't use public knowledge about the buyer

Cost & Sustainability

A weekly review of 50 deals costs about a rupee in AI calls, so cost is not the constraint; trust and accuracy are.

Rough estimate per weekly run (approximate; check the provider's current pricing):

- About 400 input tokens per deal (stage, dates, size, notes, instructions) and 150 output tokens
- 50 deals means roughly 20,000 input and 7,500 output tokens
- At Groq's paid rates for openai/gpt-oss-120b ($0.15 per million input tokens and $0.60 per million output tokens), that is under $0.01, or less than ₹1 per run
- 100 managers running it weekly would cost a few hundred rupees a month

The prototype costs nothing to run. It uses Groq's free tier, which needs no card and refuses requests over its limits instead of charging. The tight limit is tokens per minute: several analyses in a row, or a very large upload, can hit it. When that happens the app falls back to rules-only results, and waiting a minute fixes it.

How I keep it cheap and stable:

- **Rules run first.** Deals with no notes and no rule hits never reach the AI.
- **Skip unchanged deals.** Not built yet: reusing the stored result when a deal hasn't changed since the last run would cut AI calls further.
- **Batch calls.** One API call per run instead of one per deal.
- **Rules-only fallback.** If the API is down or the budget is hit, the tool still works without the AI layer.

Hosting: Replit for the prototype (the free Starter plan includes one published app), with SQLite for storage. A real deployment would move to a managed database and add login per sales team.

Reflection

The most useful decision in this brief was shrinking the AI's job.

My first idea was to paste the pipeline into an LLM and ask which deals were at risk. Working through the failure modes showed why that fails: the model has no fixed definition of "stalled", it would answer differently each week, and it would flag deals just to look thorough. A manager would stop trusting it quickly.

Splitting the work changed the product. Rules now carry the score, which makes it explainable and testable. The AI does what only it can do, reading messy notes, and it must show its evidence. The outcome tracking matters more than I first thought, because it is the only way to prove the tool flags the right deals.

What building it showed. Before writing any code I set the expected result for each of the 7 test deals. The rules alone matched all six scored deals, and the seventh, with a future activity date, was rejected. With the AI switched on, the final scores matched again.

The deal I learned most from was D006. It had 21 quiet days, so the rules put it on Watch. Its notes said the contract was with the buyer's legal team and a reply was due on 20 Oct. The AI quoted that line and the deal dropped to Healthy. That is the false-positive safeguard working on a realistic case, and it only works because the AI has to show the exact words it relied on.

The build also tested the failure handling for real. The first Groq model I chose wasn't available on my account, and the app

showed rules-only results with the error in a banner instead of crashing. Switching models took one setting.

What is still unproven. Seven sample deals show the logic works as designed, but not that the thresholds are right. That needs a few weeks of real outcomes on the dashboard, measured against the 60% accuracy and 20% override targets. Next I would let managers edit the thresholds in the app, then add CRM import so the weekly upload goes away.